

# CHAINTRACK — MASTER PROJECT CONTEXT

## Blockchain Supply Chain Tracker

This document is the **single handoff context** for future development threads. A new thread should be able to read it and understand the project without relying on previous conversation history.

**\*\*Locked direction:\*\* ChainTrack is a professional, light-themed, fast-rendering blockchain supply-chain tracker using **\*\*Navy + Orange\*\***, purposeful animations, a hybrid blockchain/PostgreSQL architecture, and an MVP-first implementation strategy.**

## 1. Project Identity

**Project:** ChainTrack – Blockchain Supply Chain Tracker

**Core idea:** Products move through a supply chain. ChainTrack gives each product a unique digital identity, records its lifecycle and custody changes on blockchain, and lets anyone scan a QR code to verify authenticity and view the journey.

**Primary goal:** Demonstrate a meaningful real-world use of blockchain technology—not merely storing arbitrary data on-chain.

**Primary audience:** Academic/project evaluation, with a technically defensible architecture and a reliable live/local demo.

**Build principle:** MVP first → local Hardhat reliability → testing → Sepolia public demo → stretch features.

## 2. Problem

- Supply-chain records are fragmented between manufacturers, distributors/transporters and retailers.
- Records can be edited, lost or disputed.
- Fake or tampered products can enter the chain because there is no single trusted historical record.
- Customers cannot easily verify where a product came from or whether it is genuine.

## 3. Solution

ChainTrack creates a shared, tamper-resistant product journey.

1. Admin registers/approves participants.
2. Manufacturer registers a product.
3. Product receives a unique blockchain ID and QR code.
4. Manufacturer transfers the product to a Distributor.
5. Distributor accepts/rejects and can update location.
6. Distributor transfers to Retailer.
7. Retailer accepts and eventually marks the product Sold.
8. Customer scans the QR without logging in.
9. ChainTrack verifies the blockchain record and the integrity of the off-chain data.
10. Customer sees **Authentic**, **Data Mismatch**, or **Not Found**.

This is **not** a payment or cryptocurrency project. Real payments, invoicing and crypto transfers are out of scope.

## 4. Project Goals

- Give every product a unique digital identity and QR code.
- Record every custody change and status update immutably on blockchain.
- Let customers verify authenticity and the full journey without login.
- Restrict actions by role and current ownership.
- Deliver a complete, demo-ready end-to-end system.

## 5. Users and Roles

### Admin

- Wallet login.
- Contract owner/deployer.
- Registers and approves Manufacturer, Distributor and Retailer.
- Activates/deactivates participants.

### Manufacturer

- Wallet login.
- Creates products.
- Generates product QR.
- Owns product initially.
- Initiates transfer to Distributor.

### Distributor

- Wallet login.
- Receives products.
- Accepts/rejects transfers.
- Updates location while holding product.
- Initiates transfer to Retailer.

### Retailer

- Wallet login.
- Receives products.
- Accepts/rejects transfers.
- Can update location where allowed.
- Marks product Sold.

### Customer

- No login.
- Scans QR or enters Product ID.
- Views authenticity and complete product journey.

## 6. Critical Business Rules

These rules are core invariants and should not be changed casually.

- Only registered and active participants can perform actions.
- Only the current owner can initiate a transfer or update location.
- Only the designated pending receiver can accept or reject.
- Ownership order is fixed: \*\*Manufacturer → Distributor → Retailer → Sold\*\*.

- Blockchain records cannot be edited or deleted.
- Corrections are represented by new events.
- Sold is a final state.
- Contract-level permissions are the real security boundary; hiding frontend buttons is not sufficient.

## 7. Product State Machine

Created

↓

InTransit

↓

AtDistributor

↓

InTransit

↓

AtRetailer

↓

Sold (FINAL)

**Created:** Manufacturer registered product and still owns it.

**InTransit:** Current owner initiated a handover; receiver has not accepted.

**AtDistributor:** Distributor accepted and owns the product.

**AtRetailer:** Retailer accepted and owns the product.

**Sold:** Retailer sold to customer; no future state changes.

### Two-step transfer

1. Current owner initiates transfer to a named receiver.
2. Status becomes InTransit.
3. Receiver accepts → ownership/status changes.
4. Receiver rejects → product returns to sender and previous status.
5. The rejection itself remains in the immutable history.

## 8. Hybrid Storage — Central Architecture Decision

Blockchain is **not** used as a normal database for everything.

### On-chain — authoritative

- Product ID
- Current owner
- Current status
- Participant wallet/role/active state

- Event history
- Timestamps
- Locations
- Product data hash

## PostgreSQL — off-chain

- Product name
- Description
- Images
- Organization details
- Contact information
- Search indexes
- Dashboard statistics
- Authentication nonces
- Indexed event/search data
- Draft metadata

## Source-of-truth rule

**Blockchain wins** for product existence, owner, status, event history and data hash.

**Database wins** for descriptive/profile/search data.

The public verification page should read blockchain status/history directly so verification can still show chain-derived information when PostgreSQL is unavailable.

# 9. Why the Hash Exists

The full product data is stored off-chain because storing large descriptive data on-chain is inefficient.

A deterministic/canonical representation of the product data is hashed.

Product metadata

↓

Canonicalize

↓

Compute dataHash

↓

Store metadata in PostgreSQL

↓

Store dataHash on blockchain

During verification:

Current PostgreSQL data

↓

Recompute hash

↓

↓

Mismatch → Data Mismatch

[illegible][illegible]

▼

[illegible]

© 2013 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., 501 Boylston Street, Boston, MA 02116

▼

[illegible]

<https://doi.org/10.1016/j.jmb.2020.105400>

■

▼

© 2015 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10984.

Journal Pre-proof

- Smart contract: **\*\*Solidity ^0.8.24\*\***
- Contract framework: **\*\*Hardhat + TypeScript\*\***
- Local blockchain: **\*\*Hardhat node, chain ID 31337\*\***
- Public testnet: **\*\*Sepolia, chain ID 11155111\*\***

- Web + backend: **Next.js App Router + TypeScript**
- Styling: **Tailwind CSS**, shadcn/ui optional
- Blockchain client: **ethers.js v6**
- Wallet: **MetaMask**
- Database: **PostgreSQL**
- ORM: **Prisma**
- Validation: **Zod**
- QR generation: **qrcode**
- QR scanning: **html5-qrcode** or equivalent
- Authentication: **JWT** in httpOnly cookie + **jose**
- Contract tests: **Hardhat + Chai**
- Utility/API tests: **Vitest**
- E2E: **Playwright**, recommended
- Local PostgreSQL: **Docker Compose**
- Optional hosting: **Vercel + Supabase/Neon**

Use current stable package versions at implementation time and lock them in package.json.

## 12. Smart Contract

Contract name:

SupplyChain.sol

Solidity:

^0.8.24

Current design:

- Own `admin` variable.
- No payable functions.
- No Ether handling.
- No need for OpenZeppelin in the current design.
- Optimizer enabled, 200 runs.
- `viaIR` only if needed for stack-depth issues.

### Enums

#### Role

- None
- Manufacturer
- Distributor
- Retailer

#### Status

- Created
- InTransit
- AtDistributor
- AtRetailer
- Sold

### EventType

- Registered
- TransferInitiated
- TransferAccepted
- TransferRejected
- LocationUpdate
- Sold

## Structs

### Participant

- role
- active

### Product

- id
- dataHash
- manufacturer
- currentOwner
- pendingReceiver
- status
- createdAt

### HistoryEntry

- eventType
- actor
- counterparty
- timestamp
- location
- note

## Storage

- admin
- productCount
- participants[address]
- products[id]
- histories[id][]
- hashUsed[dataHash]

Limits:

- Location: 100 bytes.
- Note: 280 bytes.

# 13. Core Contract Functions

### registerParticipant(wallet, role)

- Admin only.
- Wallet must be non-zero.
- Role cannot be None.

- Wallet must not already have a role.

#### **setParticipantActive(wallet, active)**

- Admin only.
- Participant must exist.

#### **registerProduct(dataHash, location)**

- Manufacturer only.
- Hash must be non-zero and unused.
- Creates product in Created state.
- Current owner = manufacturer.
- Adds Registered history.

#### **initiateTransfer(id, to, location, note)**

- Current owner only.
- Receiver must be the correct next role.
- Product must be in a valid holding state.
- Moves product to InTransit.

#### **acceptTransfer(id)**

- Pending receiver only.
- Transfers ownership.
- Moves to AtDistributor or AtRetailer.

#### **rejectTransfer(id)**

- Pending receiver only.
- Restores sender and previous status.
- Records rejection history.

#### **addLocationUpdate(id, location)**

- Current owner only.
- Only valid holding states.

#### **markSold(id)**

- Retailer only.
- Product must be AtRetailer.
- Sold is final.

Every state-changing action should emit an event and preserve append-only history.

## **14. Wallet Authentication**

Authentication is wallet-signature based.

Connect Wallet

↓

Check network

↓

Request one-time nonce

↓

MetaMask signs nonce

↓

Server verifies signature

↓

Read participant role/active from contract

↓

Create httpOnly JWT session

↓

Redirect to role dashboard

Rules:

- Nonce is single-use.
- Nonce expires after 5 minutes.
- Wrong network → Switch Network UI before signature.
- Server recovers signer and validates nonce.
- Role/active state comes from contract.
- Unregistered/inactive wallet gets a contact-admin message.
- Admin is recognized through the contract owner/deployer address.

## 15. Participant Onboarding

Admin wallet

↓

Participants → Add participant

↓

Wallet + role + organization/contact details

↓

MetaMask transaction

↓

Contract registry

↓

Indexer

↓

Participant can log in

Participants should be deactivated rather than deleted so historical attribution remains meaningful.

## 16. Product Registration

Product form

↓

Canonicalize metadata

↓

Compute dataHash

↓

Save off-chain metadata

↓

Manufacturer signs registerProduct()

↓

Blockchain creates Product ID

↓

Indexer confirms

↓

Generate QR

Important distinction:

- `chainProductId` = blockchain identity.
- `serialNumber` = product/business identifier in the database.

QR should point to a public verification route such as:

/verify/{productId}

## 17. Transfer Lifecycle

### Manufacturer → Distributor

Manufacturer selects active Distributor and initiates transfer.

Distributor can:

- Accept.
- Reject.

### Distributor → Retailer

Distributor owns product after acceptance and can:

- Update location.
- Initiate transfer to Retailer.

Retailer can:

- Accept.
- Reject.

### Retail Sale

Retailer marks AtRetailer product as Sold.

Sold is final.

## 18. Public Verification

Customer experience must be extremely simple.

Scan QR

↓

Public verification page



Read blockchain state/history



Fetch descriptive data



Recompute hash



Compare



Authentic / Data Mismatch / Not Found

### Authentic

Product exists on blockchain and off-chain hash matches.

### Data Mismatch

Product exists on blockchain, but current off-chain data differs from recorded hash.

### Not Found

Product ID does not exist on blockchain.

Recommended page:

- Large authenticity result.
- Product name/ID/serial number.
- Product image.
- Current status.
- Current owner.
- Supply-chain timeline.
- Event timestamps.
- Actors/counterparties.
- Locations.
- Transaction hashes.
- Expandable blockchain proof.

No login required.

## 19. Event Indexer

The indexer makes blockchain data fast and searchable without becoming the source of truth.

deployBlock



Read blockchain logs



Process event



Store ProductEvent / Participant change

↓

Update product snapshot

↓

Persist lastProcessedBlock

↓

Repeat

Requirements:

- Start from `deployBlock`.
- Persist `lastProcessedBlock`.
- Idempotent processing.
- Use transaction hash + log index or equivalent unique event identity.
- Never delete blockchain-derived history.
- Do not incorrectly recreate the blockchain state machine in the backend.

`ProductEvent` logically links to Product through `chainProductId`.

## 20. Database Entities

### Participant

- id
- walletAddress
- role
- organizationName
- isActive
- profile/contact fields

### Product

- id
- chainProductId
- serialNumber
- dataHash
- name
- currentStatus
- currentOwner
- isConfirmed
- descriptive metadata

### ProductEvent

- id
- chainProductId
- eventType
- txHash
- logIndex
- event details

### AuthNonce

- address
- nonce
- expiresAt

### IndexerState

- lastProcessedBlock

## 21. Final UI/UX Direction

The selected visual direction is:

### Navy + Orange

Primary:

- Navy `#173B57`
- Orange `#F97316`

Supporting:

- Navy Light `#EAF1F5`
- Orange Light `#FFF1E8`
- Background `#F8FAFC`
- Surface `#FFFFFF`
- Text `#17202A`
- Muted `#64748B`
- Border `#E2E8F0`

Semantic:

- Verified/Sold `#16A34A`
- Pending/In Transit `#F59E0B`
- Error/Mismatch/Rejected `#DC2626`

### UX principles

- Light and professional.
- Fast rendering.
- Responsive.
- Clean information hierarchy.
- Blockchain details available as proof without overwhelming normal users.
- Strong transaction feedback.
- Clear pending/success/failure states.
- QR workflow optimized for mobile.

### Animation principles

Animations are required, but speed comes first.

Use:

- Transform/opacity transitions.
- Short micro-interactions.
- Timeline transitions.
- Transaction progress feedback.
- Page/state transitions.
- Skeleton loading.

Avoid:

- Heavy particle backgrounds.
- Constant decorative animations.
- Large 3D/WebGL effects.
- Excessive motion.

## 22. Main Screens

### ### Admin

- Dashboard
- Participants
- Products/Activity
- Settings

### ### Manufacturer

- Dashboard
- My Products
- Create Product
- Transfers
- History

### ### Distributor

- Dashboard
- Incoming
- Inventory
- Shipments
- History

### ### Retailer

- Dashboard
- Incoming
- Inventory
- Sold Products
- History

### ### Customer

- Public Verify page.

Reusable components:

- App shell.
- Wallet connection.
- Product card/detail.
- Status badge.
- Role badge.
- Timeline.
- Transfer card.
- Transaction modal.
- QR generator/download.
- QR scanner.
- Search/filter/pagination.
- Toast.

- Confirmation modal.
- Error/empty/skeleton states.

## 23. MVP — Must Ship

1. Wallet login and role access.
2. Admin participant management.
3. Product registration.
4. QR generation.
5. Manufacturer → Distributor transfer.
6. Distributor → Retailer transfer.
7. Accept/reject workflow.
8. Location updates.
9. Retailer marks Sold.
10. Public verification.
11. Role dashboards.
12. Search/filter.

## 24. MVP Acceptance Criteria

The MVP is complete only when:

1. Admin registers one Manufacturer, one Distributor and one Retailer.
2. Manufacturer registers a product and downloads its QR.
3. Product moves Manufacturer → Distributor → Retailer using initiate/accept and reaches Sold.
4. Wrong-role/wrong-owner action is rejected clearly.
5. Reject returns product to sender correctly.
6. QR scan without login shows Authentic with complete ordered timeline.
7. Direct modification of off-chain product data causes Data Mismatch.
8. Unknown product ID shows Not Found.
9. Entire flow runs on local chain with one setup guide.

## 25. Implementation Phases

### Phase 0 — Design System

- Colors.
- Typography.
- Components.
- Screens.
- Animation/performance rules.

### Phase 1 — Environment

- Repository.
- Hardhat.
- Next.js.

- PostgreSQL.
- Prisma.
- Environment variables.
- Scripts.

## **Phase 2 — Smart Contract**

- Roles.
- Product.
- State machine.
- Transfers.
- History.
- Location.
- Sold.
- Events.

## **Phase 3 — Contract Testing**

- Positive transitions.
- Negative transitions.
- Permissions.
- Rollbacks.
- Final state.
- Event order.

## **Phase 4 — Backend + DB**

- Prisma schema.
- API routes.
- Wallet nonce.
- Signature verification.
- Validation.
- Metadata/upload support.

## **Phase 5 — Indexer**

- Event listener/poller.
- Block cursor.
- Idempotency.
- Product snapshots.
- Participant mirroring.

## **Phase 6 — Frontend + Auth**

- App shell.
- MetaMask.
- Network switching.
- Signature login.
- JWT.
- Role routing.

## **Phase 7 — Admin**

- Participant registration.
- Activation/deactivation.
- Participant dashboard.

## **Phase 8 — Product + QR**

- Product form.

- Canonical hash.
- Product registration.
- Confirmation.
- QR generation.

#### **Phase 9 — Transfers**

- Manufacturer transfer.
- Distributor accept/reject.
- Location.
- Distributor → Retailer.
- Retailer accept/reject.
- Sold.

#### **Phase 10 — Verification**

- QR scan.
- Blockchain read.
- Metadata read.
- Hash comparison.
- Authentic/Mismatch/Not Found.

#### **Phase 11 — Hardening**

- Security.
- Performance.
- Responsive UI.
- Loading/error states.
- E2E testing.
- Demo polish.

#### **Phase 12 — Deployment**

- Local freeze.
- Sepolia contract.
- Database deployment.
- Indexer.
- Public web app.
- Final demo.
- Local fallback retained.

## **26. Testing Strategy**

### **### Smart Contract**

Must test:

- Admin-only participant management.
- Every valid transition.
- Every invalid transition.
- Wrong role.
- Wrong owner.
- Wrong receiver.
- Accept/reject permissions.
- Reject rollback.
- Sold finality.

- Events/history ordering.

Target:

- 100% write functions covered.
- At least 90% line coverage.

### Backend / Utilities

Test:

- Deterministic product hash.
- Nonce expiry.
- Nonce reuse rejection.
- Authentic.
- Data Mismatch.
- Not Found.
- Indexer idempotency.

### E2E

Recommended:

- Playwright full MVP flow on local chain.

## 27. Security Requirements

- No user private keys in frontend/backend.
- Deployer key only in environment variables.
- Contract-level access control on all writes.
- Custom errors.
- No payable functions/external calls in current contract.
- Zod validation.
- JWT protection.
- Rate limiting for auth/verification.
- Single-use expiring nonce.
- httpOnly cookie.
- Upload type/size checks.
- Random upload filenames.
- Restricted CORS.
- Security headers.
- XSS-safe output.
- Prisma/parameterized database access.
- No secrets in logs.

## 28. Performance Targets

- Dashboard/list load: **under 3 seconds**.
- Verification: **under 5 seconds on Sepolia**.
- Verification: **under 1 second locally**.
- Indexer lag: **under 30 seconds on Sepolia**.
- Pagination: **20 items per page**.
- Database outage: public verification still shows chain-derived data.
- RPC outage: friendly retry UI.

Animations must never compromise these goals.

## 29. Deployment Strategy

### Local

- Hardhat node.
- Chain ID 31337.
- Docker PostgreSQL.
- Primary development/demo environment.
- Reliable fallback.

### Sepolia

- Chain ID 11155111.
- Final public demo/submission.
- Supabase or Neon PostgreSQL.
- Never use Hardhat default private keys.

Repository:

```
chaintrack/  
■■■■ contracts/  
■ ■■■■ contracts/  
■ ■■■■ test/  
■ ■■■■ scripts/  
■ ■■■■ hardhat.config.ts  
■■■■ web/  
■ ■■■■ src/app/  
■ ■■■■ src/lib/  
■ ■■■■ src/lib/contract/  
■ ■■■■ src/components/  
■ ■■■■ prisma/  
■■■■ docker-compose.yml  
■■■■ docs/  
■■■■ README.md
```

Scripts:

```
npm run chain  
npm run deploy:local  
npm run seed  
npm run dev  
npm run test
```

Demo setup should be documented to take under 15 minutes.

## 30. Local Demo Seed

Suggested local Hardhat accounts:

- #0 — Admin/deployer.
- #1 — Manufacturer, “Acme Manufacturing”.
- #2 — Distributor, “FastMove Logistics”.
- #3 — Retailer, “CityMart Retail”.
- #4 — Extra Distributor for rejection demonstrations.

Seed five sample products, ideally one in each state:

- Created
- InTransit
- AtDistributor
- AtRetailer
- Sold

Hardhat default keys are public and must never be used on Sepolia or a real network.

## 31. Risks & Mitigations

### Wallet/network confusion

→ One network at a time, clear switch-network prompts, local fallback.

### Scope creep

→ MVP locked; stretch only after acceptance criteria.

### Testnet slowness/faucet limits

→ Local development/demo; Sepolia as final step.

### On-chain/off-chain mismatch

→ Hash comparison + blockchain source of truth.

### Indexer failure

→ Block cursor, idempotency, retry strategy; public verification reads chain.

### Heavy animation

→ Transform/opacity and short transitions; avoid continuous heavy effects.

## 32. Stretch Features — Only After MVP

Possible future features:

- Batch registration.
- PDF authenticity certificate.
- Suspicious scan alert.
- Recall flag.
- Map view of journey.
- Analytics charts.

Do not let stretch features destabilize the core state machine.

## 33. Examiner Explanation

### One-sentence answer

“ChainTrack is a blockchain-based supply-chain tracker that records product ownership and lifecycle events immutably, while a QR code lets anyone verify the product’s authenticity and journey.”

### Technical explanation

- Smart contract stores trusted state/history.
- Participants sign actions through MetaMask.
- Every product has a blockchain ID and data hash.
- PostgreSQL stores descriptive/search data.
- Indexer mirrors blockchain events for fast application queries.
- Customers need no wallet.
- Verification compares off-chain data against the immutable on-chain hash.

### Why blockchain is actually useful

The blockchain is not being used simply as another database. It provides a shared, tamper-resistant history between independent supply-chain parties. No single participant should be able to silently rewrite historical custody events.

## 34. Future-Thread Handoff Protocol

Before proposing code or architecture in a new thread, read this document.

The new thread should already know:

- Project purpose.
- Problem and solution.
- Roles.
- Permissions.
- Lifecycle.
- Hybrid storage.
- Smart contract.
- Authentication.
- Indexer.
- Database.
- Verification.
- UI theme.
- Animation philosophy.
- MVP.
- Acceptance criteria.
- Implementation phases.
- Security.
- Performance.
- Deployment.
- Risks.
- Non-goals.

## Change-control rule

If a future thread proposes changing:

- Lifecycle/state machine.
- Role permissions.
- On-chain/off-chain boundary.
- Authentication model.
- Contract identity.
- MVP acceptance criteria.

...it must explicitly explain why the change is necessary, what existing behavior it affects, and what tests/flows must be updated.

## 35. Source Authority

The project is based on these detailed documents:

1. **01-PRD.md** — product requirements, problem, goals, roles, lifecycle, business rules, NFRs, MVP/stretch and acceptance criteria.
2. **02-TRD.md** — technical stack, architecture, security, performance, testing, deployment and repository.
3. **03-System-Flow.md** — login, onboarding, product registration, transfers, verification, indexer, state machine, role/action matrix, navigation and edge cases.
4. **04-Schema-and-Contract-Design.md** — smart contract types/storage/functions/events, PostgreSQL schema, indexer, deployment and seed.
5. **Implementation Plan** — detailed build phases.
6. **This Master Project Context** — consolidated handoff reference plus finalized UI/UX/project decisions.

If this summary ever conflicts with a detailed source document or a later explicitly approved decision, inspect the detailed source and latest approved decision before changing implementation.

## FINAL PROJECT PRINCIPLE

**Build the trust layer first.**

Smart contract → tests → database/indexer → wallet/auth → product registration → transfers → public verification → polish → Sepolia.

The most important demonstration is not the dashboard.

It is:

**Create → Transfer → Receive → Transfer → Receive → Sell → Scan QR → Verify Authenticity.**

That complete journey is the heart of ChainTrack.